

Industrial OPC-UA Based Smart Factory Condition Monitoring System

Faysal Ahammed Tonmoy, MD Rafiul Islam, MD Azadul Islam, MD Jubair Salehin Razin

Team B3
Advanced Embedded Systems Lab
Hochschule Hamm-Lippstadt

Abstract—This paper presents a smart-factory condition monitoring system that connects heterogeneous Arduino-based sensor nodes to a Raspberry Pi industrial gateway and exposes the processed data through an OPC UA server. The implementation uses three different field-level communication protocols to demonstrate a practical Industrial Internet of Things (IIoT) gateway: HTTP POST for a DHT11 temperature and humidity node, MQTT for a flame detection node, and UDP for an MPU6050 vibration node. The Raspberry Pi receives the raw JSON payloads, stores the latest values in a shared gateway memory, applies status classification rules, and publishes a unified OPC UA information model containing *TemperatureNode*, *FlameNode*, *VibrationNode*, and *GatewayProcessedData*. A browser-based dashboard and a UaExpert client can read the OPC UA variables and visualize the live industrial condition. To extend the system above the gateway layer, a software PLC, SQLite/CSV historian, MES layer, and ERP live report were implemented as independent OPC UA clients that read the live sensor data and write their results back into the same address space. The resulting prototype demonstrates the complete flow from sensor-level protocols to industrial information modeling, monitoring, control decisions, historical storage, production metrics, and enterprise-level reporting.

Index Terms—Industrial IoT, OPC UA, Raspberry Pi, Arduino, HTTP, MQTT, UDP, Smart Factory, Condition Monitoring, Software PLC, MES, ERP.

I. INTRODUCTION

Modern production systems require continuous monitoring of machine and environmental conditions. Temperature, flame, and vibration are common indicators of abnormal operation: high temperature can indicate overheating, flame detection can indicate a safety-critical event, and excessive vibration can indicate mechanical wear or imbalance. In a classical factory, these signals are often connected directly to industrial controllers. In an IIoT-based system, however, small embedded nodes may communicate through different lightweight protocols before the data is converted into a standard industrial interface.

This project implements a complete laboratory prototype for such a conversion path. Three Arduino-based sensor nodes measure different physical quantities and send their data to a Raspberry Pi gateway. The temperature node uses HTTP POST, the flame node uses MQTT, and the vibration node uses UDP. The Raspberry Pi hides these protocol differences and exposes the processed data to industrial clients using OPC UA. Therefore, the main engineering problem is not only

sensor reading, but also protocol integration, status processing, data modeling, and reliable presentation to higher-level applications.

The project follows the industrial data flow shown in Fig. 1: field devices first send raw measurements, the gateway converts them to structured OPC UA variables, and clients such as UaExpert, a web dashboard, PLC software, a historian, MES, and ERP modules consume the same industrial data model. The focus of this report is the implemented system architecture, the protocol-specific data paths, the gateway logic, the OPC UA model, and the observed results.

II. SYSTEM REQUIREMENTS AND DESIGN GOALS

The system was designed as a compact smart-factory monitoring demonstrator. Its functional requirements are summarized as follows:

- acquire temperature and humidity from a DHT11 node;
- acquire flame raw value and flame-detected state from an analog flame sensor;
- acquire vibration information from an MPU6050 inertial sensor;
- support different communication protocols at the sensor layer: HTTP, MQTT, and UDP;
- convert all sensor data into a common OPC UA information model;
- classify each measurement into industrial status states such as *NORMAL*, *WARNING*, *CRITICAL*, and *ALARM*;
- show live values in an OPC UA dashboard;
- provide upper-layer processing using a software PLC, historian, MES, and ERP report operating on the live OPC UA data.

The main non-functional requirements are simplicity, explainability, and demonstrability. The project is intended for an embedded-systems and industrial-communication laboratory, so every data path must be observable in terminal logs and on the dashboard. For that reason, JSON payloads were selected at the field layer, and clear status strings were used in the OPC UA variables. Security mechanisms such as certificate-based OPC UA authentication were not enabled because the purpose of the prototype is protocol integration and condition monitoring, not production deployment.

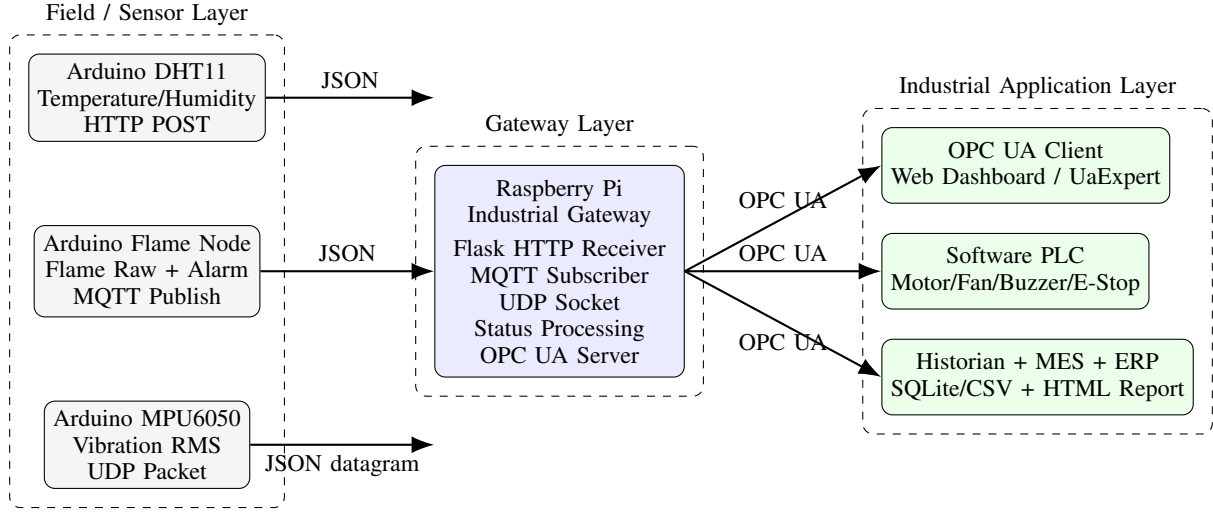


Fig. 1. Overall system architecture. Three field protocols are converted by the Raspberry Pi gateway into one OPC UA information model for industrial clients and higher-level applications.

III. HARDWARE AND SOFTWARE PLATFORM

A. Hardware Components and Pin Connections

The hardware layer consists of three independent Arduino WiFi sensor nodes and one Raspberry Pi gateway. Each Arduino node is responsible for one physical sensor only. The temperature node uses a DHT11 sensor, the flame node uses an analog flame sensor module, and the vibration node uses an MPU6050 inertial measurement unit. The Arduino boards are not connected to each other by wires. Instead, each Arduino communicates wirelessly with the Raspberry Pi through the same WiFi network. Therefore, the system is physically distributed, while the Raspberry Pi acts as the central data aggregation and protocol-conversion gateway.

The wiring was selected according to the sensor interface type. The DHT11 temperature sensor uses one digital data pin, the flame sensor uses one analog input pin, and the MPU6050 uses the I²C bus. The Raspberry Pi does not receive sensor signals directly through GPIO pins. It receives the already processed sensor payloads over WiFi using HTTP, MQTT, and UDP.

B. Interconnection Between Arduino Nodes and Raspberry Pi

There is no direct wired connection between the three Arduino boards. The Arduino nodes are independent sensor nodes. Each node has its own sensor, its own Arduino program, and its own network communication method. The only common connection between the nodes is the WiFi network. All Arduino boards and the Raspberry Pi must be connected to the same hotspot or local WiFi network.

This means that the pins used on one Arduino do not conflict with the pins used on another Arduino. For example, the DHT11 node uses digital pin D2, the flame node uses analog pin A0, and the vibration node uses SDA/SCL. These pins are located on different Arduino boards, so they are electrically independent.

The Raspberry Pi IP address used in the Arduino programs is 10.153.76.64. Therefore, all Arduino sensor nodes send their data to the same Raspberry Pi gateway, but through different protocols. The temperature node sends HTTP data to port 5000, the flame node publishes MQTT messages to the broker on port 1883, and the vibration node sends UDP packets to port 6000. The Raspberry Pi then converts these different incoming data streams into one OPC UA information model.

C. Purpose of Each Pin Connection

The DHT11 temperature node uses only one digital signal line because the DHT11 protocol sends both temperature and humidity through the same data pin. In the implemented Arduino code, this signal pin is defined as `DHTPIN 2`. The DHT library handles the timing-sensitive reading of the DHT11 signal.

The flame sensor node uses the analog output pin A0 because the project needs the raw flame intensity value. The Arduino reads this value using `analogRead(A0)`. The code then compares the raw value with the threshold `FLAME_THRESHOLD`. If the raw value is below the threshold, the code marks flame detection as true. The digital output pin of the flame module is not used because software thresholding provides more flexibility.

The MPU6050 vibration node uses the I²C bus because the MPU6050 is a register-based digital sensor. The Arduino communicates with the sensor through the `Wire` library. The code scans the I²C bus and accepts address 0x68 or 0x69. After initialization, the Arduino reads acceleration registers, calculates acceleration magnitude, removes the calibrated baseline, and computes vibration RMS before sending the result to the Raspberry Pi using UDP.

TABLE I
DETAILED SENSOR WIRING AND ARDUINO PIN CONNECTIONS

Node	Sensor / Module Pin	Arduino Connection	Code Reference	Purpose and Notes
Temperature Node	DHT11 S, DATA, or signal pin	Digital pin D2	#define DHTPIN 2	This pin carries the digital temperature and humidity data from the DHT11 module to the Arduino. The DHT library reads the value from this pin.
Temperature Node	DHT11 + or VCC	5V	Sensor power	Provides supply voltage to the DHT11 module. If a bare DHT11 sensor is used instead of a module, a pull-up resistor between data and VCC may be required.
Temperature Node	DHT11 - or GND	GND	Common reference	Provides the ground reference for the DHT11 sensor and the Arduino.
Flame Node	Flame sensor AO or analog output	Analog pin A0	const int flamePin = A0;	The analog voltage from the flame sensor is read by the Arduino ADC. A lower raw value is treated as flame detection in the implemented code.
Flame Node	Flame sensor VCC	5V	Sensor power	Provides supply voltage to the flame sensor module. Most flame sensor modules support 5 V operation.
Flame Node	Flame sensor GND	GND	Common reference	Provides the electrical ground reference for the flame sensor and the Arduino.
Flame Node	Flame sensor DO or digital output	Not used	Not used in code	The project reads the analog output only. The digital output of the module is not connected because thresholding is implemented in software using FLAME_THRESHOLD.
Vibration Node	MPU6050 SDA	Arduino SDA pin	Wire.begin()	I ² C data line. This line transfers register data between the Arduino and the MPU6050. Use the Arduino pin physically labelled SDA.
Vibration Node	MPU6050 SCL	Arduino SCL pin	Wire.begin()	I ² C clock line. This line provides the clock for communication between the Arduino and the MPU6050. Use the Arduino pin physically labelled SCL.
Vibration Node	MPU6050 VCC	3.3V preferred	Sensor power	Powers the MPU6050. Many breakout boards include a voltage regulator and can accept 5 V, but 3.3 V is the safest option for I ² C logic compatibility.
Vibration Node	MPU6050 GND	GND	Common reference	Provides the ground reference for the MPU6050 and the Arduino.
Vibration Node	MPU6050 AD0	GND or left at default low	I ² C address selection	Selects I ² C address 0x68. If connected to VCC, the address becomes 0x69. The implemented scanner checks both addresses.
Vibration Node	MPU6050 INT	Not used	Not used in code	Interrupt output is not required because the Arduino reads acceleration values periodically using I ² C polling.

TABLE II
NETWORK-LEVEL CONNECTIONS BETWEEN NODES

Source Node	Communication Target	Protocol and Port
DHT11 Temperature Arduino	Raspberry Pi Flask server	HTTP POST to /temperature, port 5000
Flame Sensor Arduino	Mosquitto broker on Raspberry Pi	MQTT publish to topic factory/flame, port 1883
MPU6050 Vibration Arduino	Raspberry Pi UDP receiver	UDP packet to port 6000
Raspberry Pi Gateway	UaExpert / dashboard / software layers	OPC UA endpoint on port 4840

D. Grounding and Power Considerations

Each Arduino sensor node must share a common ground with its own connected sensor. For example, the DHT11 ground must be connected to the ground of the Arduino that reads it. The same rule applies to the flame sensor and MPU6050. However, because the three Arduino boards

communicate with the Raspberry Pi over WiFi, there is no need to connect the ground pins of all Arduino boards together unless they are powered from a shared external supply.

The DHT11 and flame sensor modules are powered from the Arduino 5V pin. The MPU6050 is preferably powered from 3.3V, especially when using an Arduino board with 3.3 V I²C logic. If an MPU6050 breakout board with an onboard regulator and level shifting is used, it may also accept 5 V, but this depends on the specific module. For safe operation, the wiring should follow the module markings and voltage rating.

E. Summary of Physical and Logical Connections

Physically, each Arduino is connected only to its own sensor. Logically, all Arduino nodes are connected to the Raspberry Pi through WiFi. The Raspberry Pi receives the data from all nodes, stores the latest values in gateway memory, applies status logic, and publishes the result as OPC UA variables. Therefore, the wiring layer and the network layer are

separated: sensor wires stay local to each Arduino, while inter-node communication happens through WiFi and the Raspberry Pi gateway.

F. Software Components

The Raspberry Pi software is implemented in Python. Flask is used as the HTTP server for the temperature endpoint, Paho MQTT is used as the MQTT client for flame data, Python sockets are used for the UDP receiver, and `python-opcua` is used to create the OPC UA server. Additional modules implement a browser dashboard, a software PLC, a SQLite/CSV historian, a MES production layer, and an ERP live report; each of these modules is an independent OPC UA client of the same gateway server. On the Arduino side, the implementation uses `WiFiNINA`, `DHT`, `PubSubClient`, `WiFiUdp`, and `Wire` libraries.

IV. COMMUNICATION PROTOCOLS AT THE SENSOR LAYER

A. HTTP Temperature Node

HTTP is used for the DHT11 temperature and humidity node. This is suitable because temperature and humidity are slow-changing physical quantities; they do not require high-rate streaming. The Arduino reads the DHT11 values every two seconds, builds a JSON object, opens a TCP connection to the Raspberry Pi, and sends an HTTP POST request to the `/temperature` endpoint on port 5000.

A typical payload is shown in Listing 1. It contains a device identifier, a sample counter, the temperature, and the humidity. The payload is human-readable, easy to debug in terminal logs, and directly compatible with the Flask JSON parser.

Listing 1. Example JSON payload from the DHT11 temperature node.

```

1  {
2      "device": "dht11_node",
3      "sample": 341,
4      "temperature_c": 22.00,
5      "humidity_percent": 63.00
6  }
```

The Arduino HTTP request consists of a POST method, the endpoint path, the host header, the content type, the content length, and the JSON body. On the Raspberry Pi, Flask binds the endpoint using the route decorator `@app.route("/temperature", methods=["POST"])`. The handler calls the Flask helper `get_json()` with `force=True` to read the body, converts the received values safely, stores them in the shared gateway dictionary, marks the node online, updates timestamps, and recalculates both temperature and overall gateway status.

B. MQTT Flame Node

The flame node uses MQTT because flame detection is event-oriented. The Arduino reads the analog flame sensor value and compares it against a threshold. If the raw value is below the threshold, flame is considered detected. The Arduino then publishes a JSON payload to the `factory/flame` topic. A Mosquitto broker runs on the Raspberry Pi, and the gateway acts as an MQTT subscriber.

The MQTT part demonstrates publish/subscribe communication. The sensor node does not need to know which application will consume the message. It only publishes to the topic. The gateway subscribes to that topic and receives new flame messages using the MQTT callback function. This separation is useful for industrial monitoring because the number of subscribers can change without changing the publisher.

C. UDP Vibration Node

The vibration node uses UDP because vibration data can be updated more frequently and does not require the request-response overhead of HTTP. The Arduino reads the MPU6050 via I²C, calibrates a baseline, collects multiple acceleration samples, subtracts the static baseline, calculates dynamic acceleration, and computes vibration RMS. The RMS value is then converted to vibration in *g* and classified into *NORMAL*, *WARNING*, or *CRITICAL* before being sent to the Raspberry Pi UDP port 6000.

UDP is connectionless, so it is lightweight and simple. However, it does not guarantee delivery. In this prototype, occasional packet loss is acceptable because the gateway continuously receives new vibration samples and always displays the latest received state. For production systems, additional sequence counters, timestamps, or reliable transport could be added.

V. RASPBERRY PI GATEWAY IMPLEMENTATION

A. Multi-Protocol Receiver

The Raspberry Pi gateway is the central integration point of the system. It receives data from HTTP, MQTT, and UDP at the same time. This is implemented by starting the HTTP receiver, MQTT subscriber, and UDP server in separate Python threads, while the OPC UA server runs continuously and publishes the latest processed values.

The reason for using threads is that the three input protocols have different runtime behavior. Flask waits for HTTP requests, the MQTT client blocks inside its message loop, and the UDP socket blocks while waiting for datagrams. If all components were executed sequentially in one loop, one blocked communication path could delay or stop the others. With separate threads, each protocol can receive data independently.

B. Shared Gateway Memory and Locking

All received data is stored in a central Python dictionary. This dictionary contains substructures for temperature, flame, vibration, and processed gateway values. Because several threads can update or read the dictionary at the same time, the implementation uses a `threading.Lock`. The lock ensures that only one thread writes or reads the shared data at a time. This prevents inconsistent values, for example updating a temperature value while the OPC UA server is reading the same structure.

Listing 2. Simplified gateway data structure.

```

1  data = {
2      "temperature": {
```

```

3      "online": False,
4      "temperature_c": 0.0,
5      "humidity_percent": 0.0,
6      "status": "NO_DATA",
7      "last_seen": 0.0
8    },
9    "flame": {
10     "online": False,
11     "flame_raw": 0,
12     "flame_detected": False,
13     "status": "NO_DATA",
14     "last_seen": 0.0
15   },
16   "gateway": {
17     "overall_status": "NO_DATA",
18     "overall_alarm": False
19   }
20 }

```

C. Safe Conversion and Sensor Timeout

The gateway includes safe conversion helper functions such as `safe_float`, `safe_int`, and `safe_bool`. These functions avoid program crashes if a sensor sends malformed or incomplete JSON data. For example, if the temperature field is missing, the conversion function returns a default value instead of raising an exception.

A sensor timeout mechanism is also included. If a node has not sent any data for a predefined time (10 seconds in the implemented gateway), the node is marked offline and the status is changed to `NO_DATA`. This is important in industrial systems because missing data can be as relevant as wrong data. For example, a dashboard showing an old normal value would be misleading if the sensor node had already disconnected.

VI. STATUS PROCESSING LOGIC

A. Temperature Status

The temperature status is calculated from temperature and humidity. If no data is available, the node is marked `NO_DATA`. A temperature above or equal to 45°C is classified as `CRITICAL`. A temperature above or equal to 35°C or humidity above or equal to 80% is classified as `WARNING`. Otherwise, the node is `NORMAL`. This rule is simple enough to explain during demonstration while still representing typical industrial threshold monitoring.

B. Flame Status

The flame status has two main states. If flame is detected, the node is `ALARM`; otherwise it is `NORMAL`. In the Arduino flame node, flame detection is derived from the analog raw value and a threshold. In the gateway, the received Boolean value is used directly, converted safely, and written into the OPC UA variable `Flame_Detected`.

C. Vibration Status

The vibration node calculates the vibration RMS value in m/s^2 and assigns the vibration status. The thresholds used in the project are shown in Table III. The gateway forwards the received vibration status into the OPC UA model, and the same status strings are consumed unchanged by the PLC, MES, and ERP layers.

TABLE III
IMPLEMENTED STATUS THRESHOLDS

Signal	Condition	Status
Temperature	$T < 35^\circ\text{C}$ and humidity $< 80\%$	NORMAL
Temperature	$35^\circ\text{C} \leq T < 45^\circ\text{C}$ or humidity $\geq 80\%$	WARNING
Temperature	$T \geq 45^\circ\text{C}$	CRITICAL
Flame	flame detected = true	ALARM
Vibration	$\text{RMS} < 0.25 \text{ m/s}^2$	NORMAL
Vibration	$0.25 \leq \text{RMS} < 0.80 \text{ m/s}^2$	WARNING
Vibration	$\text{RMS} \geq 0.80 \text{ m/s}^2$	CRITICAL

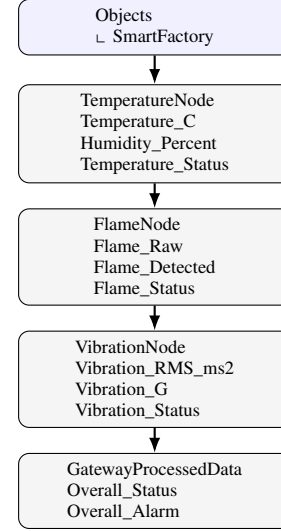


Fig. 2. Simplified OPC UA node hierarchy used by industrial clients and the dashboard.

D. Overall Gateway Status

The gateway combines all node states into one overall status. A critical temperature, flame alarm, or critical vibration produces `CRITICAL` and sets `Overall_Alarm` to true. If no critical condition exists but at least one warning or missing data condition exists, the overall status becomes `WARNING`. If all nodes are normal, the overall status is `NORMAL`. This rule gives priority to safety-critical conditions.

VII. OPC UA INFORMATION MODEL

OPC UA is used as the industrial output interface of the gateway. Instead of exposing three separate protocol-specific streams, the Raspberry Pi publishes a single structured address space. The root object is `SmartFactory`. Under this object, the gateway exposes `TemperatureNode`, `FlameNode`, `VibrationNode`, and `GatewayProcessedData` for the sensor and processing path. In addition, the gateway creates `PLCControl`, `MESProduction`, and `ERPBusinessReport` as writable objects. These are not written by the gateway itself; instead, the software PLC, MES, and ERP modules connect as OPC UA clients, read the live sensor variables, and write their own outputs back into these objects. In this way, one address space carries both the field data and the higher-level decisions.

TABLE IV
OPC UA ADDRESS SPACE USED BY THE PROTOTYPE

Object	Variable Examples	Meaning
TemperatureNode	Online, Temperature_C, Humidity_Percent, Temperature_Status, Last_Update	Environmental condition values received through HTTP.
FlameNode	Online, Flame_Raw, Flame_Detected, Flame_Status, Last_Update	Flame sensor values received through MQTT.
VibrationNode	Online, Accelerometer_Available, AX_ms2, AY_ms2, AZ_ms2, Vibration_RMS_ms2, Vibration_G, Vibration_Status	Acceleration and vibration condition values received through UDP.
GatewayProcessedData	Overall_Status, Overall_Alarm, Last_Update	Processed overall condition of the smart-factory cell.
PLCControl	Machine_State, Motor_Command, Fan_Command, Buzzer_Command, Emergency_Stop, PLC_Last_Decision	Control decisions written by the software PLC client.
MESProduction	Production_Count, Good_Count, Reject_Count, Downtime_Seconds, Availability_Percent, Performance_Percent, Quality_Percent, OEE_Percent, MES_Status	Production and performance metrics written by the MES client.
ERPBusinessReport	Estimated_Loss_EUR, Maintenance_Required, Maintenance_Ticket, Spare_Part_Status, Business_Status	Enterprise-level interpretation written by the ERP client.

The key advantage of this model is abstraction. An industrial client does not need to know that temperature arrived by HTTP, flame arrived by MQTT, and vibration arrived by UDP. It only reads typed OPC UA variables from a structured namespace. This is the central contribution of the gateway design.

VIII. DASHBOARD AND CLIENT VISUALIZATION

The web dashboard is an OPC UA client view. It connects to the OPC UA server, reads the SmartFactory variables, and stores them in a local `latest_data` structure. A Flask route `/api/data` returns these values as JSON to the browser. JavaScript refreshes the dashboard every second and updates the visible cards.

The dashboard contains four main cards: temperature node, flame node, vibration node, and gateway alarm. The overall status is displayed in a large status banner. The color changes according to the overall state: green for *NORMAL*, orange for *WARNING*, red for *CRITICAL* or *ALARM*, and grey for *NO_DATA*. This layout makes the condition of the machine cell understandable without requiring the user to inspect raw protocol logs.

IX. PLC, HISTORIAN, MES AND ERP EXTENSION

All upper layers are implemented as independent Python OPC UA clients of the gateway. They run in separate terminals, read the live sensor and gateway variables, and write their results back into the writable `PLCControl`, `MESProduction`, and `ERPBusinessReport` objects. The complete implementation architecture across all layers, including the concrete Python modules, ports, and OPC UA objects, is shown in Fig. 4. The corresponding runtime behavior of the whole chain, from sensor reading over gateway processing and PLC decisions to MES accounting and ERP reporting, is summarized in the activity diagram in Fig. 5.

A. Software PLC Layer

The software PLC reads OPC UA variables and writes machine commands back to OPC UA. Its logic is deterministic and priority-based. A flame alarm has the highest priority and triggers an emergency stop: motor off, fan off, buzzer

on, and emergency stop true. Critical vibration stops the motor, activates the buzzer, and requests maintenance. Critical temperature stops the motor, turns the fan on, and activates the buzzer. Warning states keep the motor running but enable monitoring actions, such as turning on the fan for temperature warning. Normal conditions keep the machine running. The PLC also writes its textual decision and a timestamp into `PLC_Last_Decision` and `PLC_Last_Update` so that every control action remains traceable.

B. Historian Layer

The historian reads OPC UA data once per second and writes each snapshot to both SQLite and CSV. It stores timestamps, sensor values, status strings, overall alarm, machine state, and actuator commands. This layer is important because a dashboard only shows the current value, while the historian preserves the history needed for later debugging, production analysis, or reporting.

C. MES Layer

The MES layer interprets machine state as production behavior. When the machine is running normally or with warning, production continues at an ideal rate of 12 parts per minute, approximated by a per-second probabilistic increment. During warning conditions, a part is rejected with 10% probability to model reduced process quality. When maintenance, over-temperature, or emergency stop states occur, downtime is accumulated. The MES layer calculates production count, good count, reject count, downtime, availability, performance, quality, and overall equipment effectiveness (OEE). The OEE calculation follows the standard product of availability, performance, and quality:

$$\text{OEE} = \frac{A \cdot P \cdot Q}{10000}, \quad (1)$$

where A , P , and Q are percentages. The result is also expressed as a percentage.

D. ERP Layer

The ERP layer converts MES and PLC values into business information. It estimates production loss based on downtime

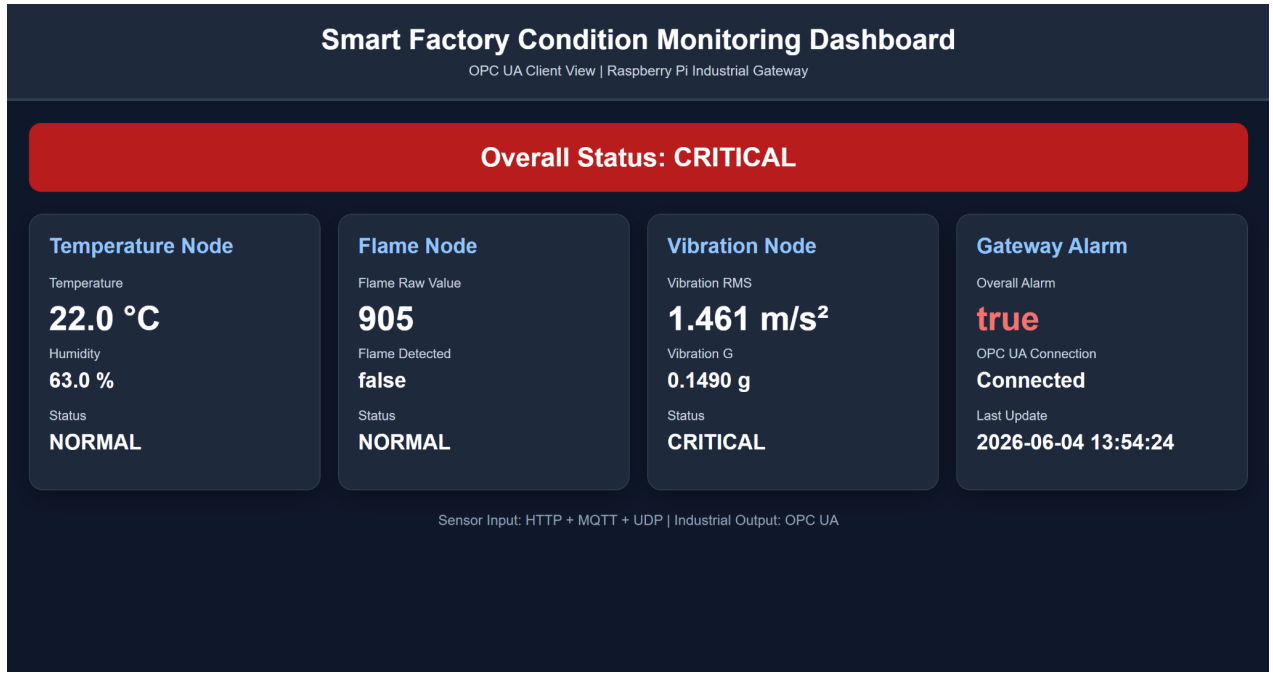


Fig. 3. Industrial OPC UA dashboard showing live sensor values. In this example, temperature and flame are normal, while vibration is critical; therefore the overall status is critical.

and a cost parameter of 2.50 EUR per downtime minute. It also creates maintenance tickets when the machine state indicates maintenance, emergency, or over-temperature, or when OEE drops below the defined lower limit of 60%. Finally, it writes business status, spare-part state, and maintenance ticket values back into OPC UA and generates an auto-refreshing HTML report.

X. TESTING AND RESULTS

A. Gateway Input Test

The first test verified that the Raspberry Pi gateway received all three protocol streams. The terminal output showed HTTP temperature messages, MQTT flame messages, and UDP vibration messages. The HTTP messages contained DHT11 samples with temperature and humidity. The MQTT messages contained flame raw values and Boolean detection results. The UDP messages contained acceleration and vibration RMS values. Fig. 9 shows the gateway receiving raw protocol data.

B. OPC UA Conversion Test

The second test verified that the gateway converted received data into OPC UA variables. The OPC UA data output showed temperature, humidity, flame raw value, flame detection, vibration RMS, vibration in g , vibration status, overall status, and overall alarm, as shown in Fig. 10. This confirmed that the gateway performed protocol abstraction and state classification.

C. Dashboard Test

The dashboard test verified end-to-end data flow from the gateway to an OPC UA client view (Fig. 3). In the shown

scenario, the temperature node is normal, the flame node is normal, and the vibration node is critical. Because vibration is critical, the overall gateway status is critical and the dashboard banner becomes red. This result demonstrates that the status-priority logic is correctly reflected in the visualization.

D. Upper-Layer Integration Test

The final test verified the complete chain on live sensor data. The software PLC (Fig. 6) reacted to flame, over-temperature, critical vibration, and warning conditions with the expected machine states and actuator commands. The historian (Fig. 7) continuously logged one snapshot per second, including the machine state and overall status, into SQLite and CSV. The MES layer accumulated production, downtime, and OEE values while the machine state changed, and the ERP layer converted these values into business status, estimated loss, and maintenance tickets in the auto-refreshing HTML report (Fig. 8). All layers operated concurrently as OPC UA clients of the same gateway server.

XI. DISCUSSION

The system demonstrates the role of a Raspberry Pi as an IIoT-to-industrial gateway. The field layer contains multiple protocols, each selected for a different sensor behavior. HTTP is simple and appropriate for slow temperature data. MQTT is suitable for event-oriented flame messages because it separates publisher and subscriber. UDP is lightweight for vibration data where continuous updates are more important than guaranteed delivery. OPC UA is then used as a unified industrial interface.

A strong point of the implementation is observability. Every layer prints readable terminal output, and the dashboard

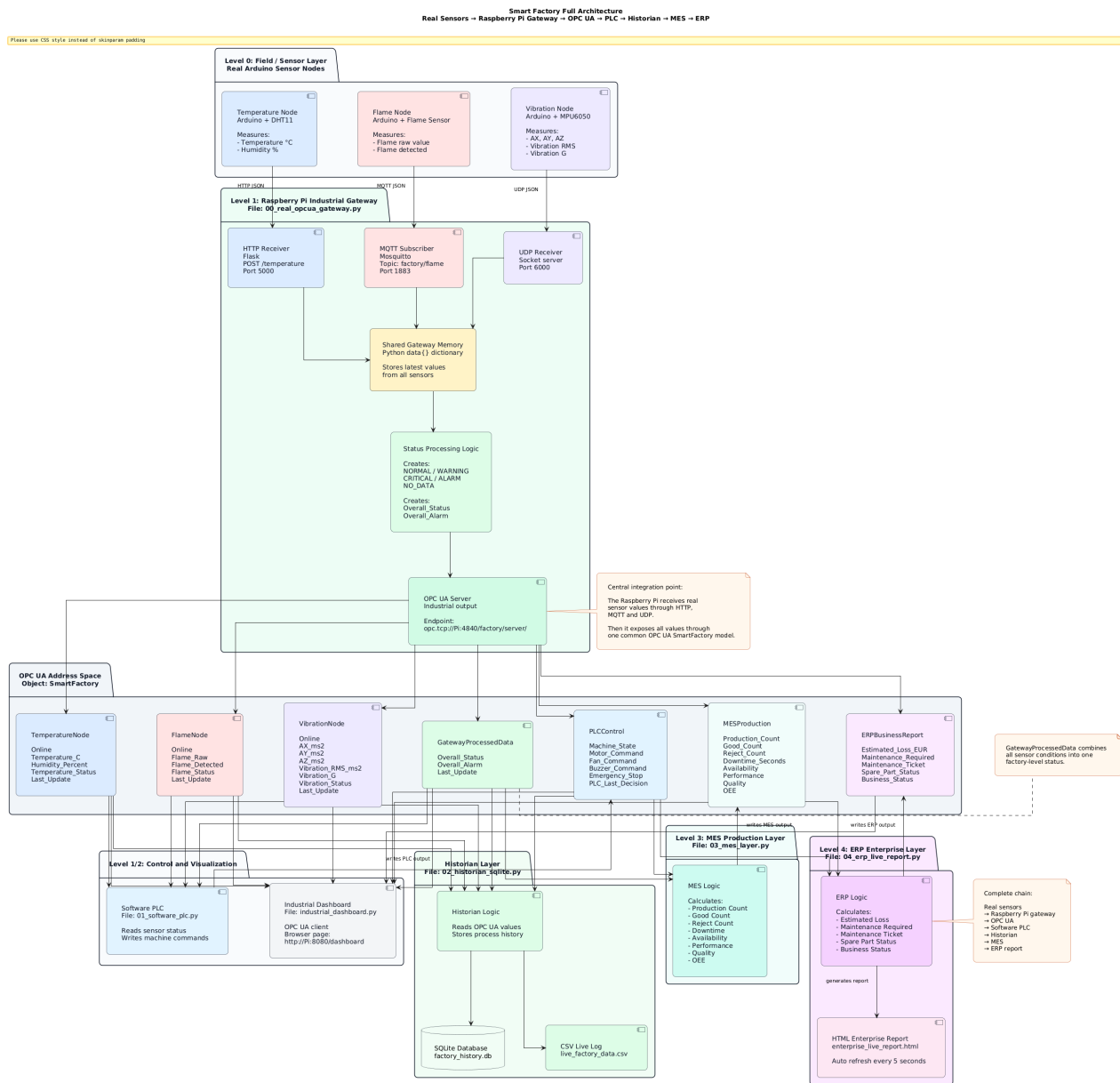


Fig. 4. Detailed implementation architecture of the complete chain. Real Arduino sensor nodes deliver data by HTTP, MQTT, and UDP to the Raspberry Pi gateway, which publishes the SmartFactory OPC UA address space; the software PLC, historian, MES, and ERP modules operate as OPC UA clients on this common model.

shows the same data in a user-friendly form. The OPC UA address space also gives a clear structure to the data, which makes it compatible with clients such as UaExpert or custom dashboards. Another strength is modularity: additional sensors can be added by creating a new receiver thread and adding new OPC UA variables, and additional software layers can be added as further OPC UA clients without changing the gateway.

The prototype also has limitations. First, the OPC UA server uses anonymous access and no security policy, which is acceptable for a laboratory demonstration but not for an industrial network. Second, the system does not implement precise

timestamp synchronization between nodes. The Raspberry Pi records receive time, but the Arduino nodes do not provide synchronized sensor timestamps. Third, UDP vibration data can be lost without retransmission. Fourth, the machine itself is virtual: the PLC commands and the MES production counts model a machine and a production process in software, so the chain validates the control, storage, and reporting layers on real sensor data, but not a full physical production line. Finally, the thresholds are fixed in code; a production system would store them in a configuration file or expose them as writable OPC UA variables.

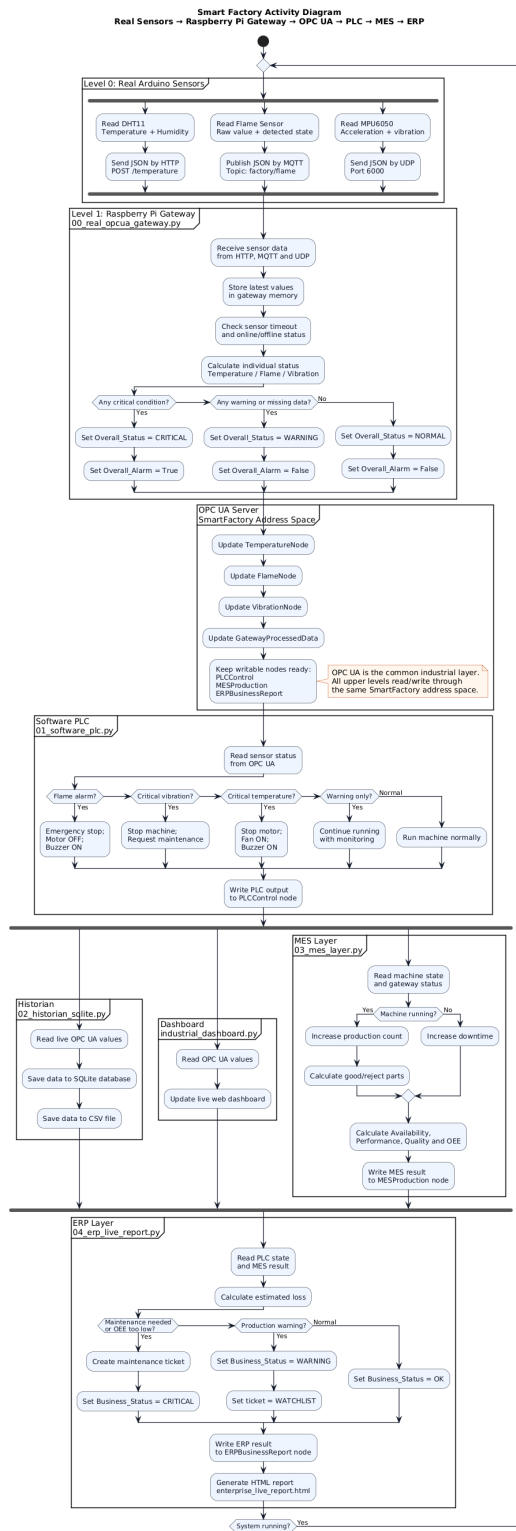


Fig. 5. Activity diagram of the complete system: sensor acquisition (Level 0), gateway processing and status classification (Level 1), OPC UA model update, PLC decision logic, and the parallel historian, dashboard, MES, and ERP activities.

```

[01 PLC OUTPUT] State=RUNNING_NORMAL | Motor=True | Fan=False | Buzzer=False | EStop=False
[01 PLC DECISION] Normal condition: machine running.

[01 PLC INPUT] Tc23.7C/NORMAL | Flame=False/NORMAL | Vib=8.115/NORMAL | Overall=NORMAL/False
[01 PLC OUTPUT] State=RUNNING_NORMAL | Motor=True | Fan=False | Buzzer=False | EStop=False
[01 PLC DECISION] Normal condition: machine running.

[01 PLC INPUT] Tc26.6C/NORMAL | Flame=False/NORMAL | Vib=8.874/NORMAL | Overall=NORMAL/False
[01 PLC OUTPUT] State=RUNNING_NORMAL | Motor=True | Fan=False | Buzzer=False | EStop=False
[01 PLC DECISION] Normal condition: machine running.

[01 PLC INPUT] Tc21.1C/NORMAL | Flame=False/NORMAL | Vib=8.889/NORMAL | Overall=NORMAL/False
[01 PLC OUTPUT] State=RUNNING_NORMAL | Motor=True | Fan=False | Buzzer=False | EStop=False
[01 PLC DECISION] Normal condition: machine running.

[01 PLC INPUT] Tc23.8C/NORMAL | Flame=False/NORMAL | Vib=8.139/NORMAL | Overall=NORMAL/False
[01 PLC OUTPUT] State=RUNNING_NORMAL | Motor=True | Fan=False | Buzzer=False | EStop=False
[01 PLC DECISION] Normal condition: machine running.

[01 PLC INPUT] Tc26.1C/NORMAL | Flame=False/NORMAL | Vib=8.883/NORMAL | Overall=NORMAL/False
[01 PLC OUTPUT] State=RUNNING_NORMAL | Motor=True | Fan=False | Buzzer=False | EStop=False
[01 PLC DECISION] Normal condition: machine running.

[01 PLC INPUT] Tc24.8C/NORMAL | Flame=False/NORMAL | Vib=8.844/NORMAL | Overall=NORMAL/False
[01 PLC OUTPUT] State=RUNNING_NORMAL | Motor=True | Fan=False | Buzzer=False | EStop=False
[01 PLC DECISION] Normal condition: machine running.

[01 PLC INPUT] Tc22.8C/NORMAL | Flame=True/ALARM | Vib=8.85/NORMAL | Overall=CRITICAL/True
[01 PLC OUTPUT] State=EMERGENCY_STOP_FLAME | Motor=False | Fan=False | Buzzer=True | EStop=True
[01 PLC DECISION] Flame alarm: stop motor, buzzer ON, emergency stop active.

[01 PLC INPUT] Tc23.7C/NORMAL | Flame=False/NORMAL | Vib=8.169/NORMAL | Overall=NORMAL/False
[01 PLC OUTPUT] State=RUNNING_NORMAL | Motor=True | Fan=True | Buzzer=False | EStop=False
[01 PLC DECISION] Normal condition: machine running.

[01 PLC INPUT] Tc38.1C/WARNING | Flame=False/NORMAL | Vib=8.884/NORMAL | Overall=WARNING/False
[01 PLC OUTPUT] State=RUNNING_WITH_WARNING | Motor=True | Fan=True | Buzzer=False | EStop=False
[01 PLC DECISION] Warning only: continue running with monitoring.

[01 PLC INPUT] Tc36.2C/WARNING | Flame=False/NORMAL | Vib=8.168/NORMAL | Overall=WARNING/False
[01 PLC OUTPUT] State=RUNNING_WITH_WARNING | Motor=True | Fan=True | Buzzer=False | EStop=False
[01 PLC DECISION] Warning only: continue running with monitoring.
  
```

Fig. 6. Software PLC terminal output showing input interpretation, machine-state decision, and actuator outputs.

```

[02 HISTORIAN] Saved row at 2020-06-13 18:00:34 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:35 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:36 | State=RUNNING_WITH_WARNING | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:00:37 | State=RUNNING_WITH_WARNING | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:00:38 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:39 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:40 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:41 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:42 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:43 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:44 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:45 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:46 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:47 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:48 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:49 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:50 | State=EMERGENCY_STOP_FLAME | Overall=CRITICAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:51 | State=EMERGENCY_STOP_FLAME | Overall=CRITICAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:52 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:53 | State=RUNNING_WITH_WARNING | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:00:54 | State=RUNNING_WITH_WARNING | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:00:55 | State=STOPPED_OVER_TEMPERATURE | Overall=CRITICAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:56 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=CRITICAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:57 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:58 | State=RUNNING_NORMAL | Overall=NORMAL
[02 HISTORIAN] Saved row at 2020-06-13 18:00:59 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:00 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:01 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:02 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:03 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:04 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:05 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:06 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:07 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:08 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:09 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:10 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:11 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:12 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:13 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:14 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:15 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:16 | State=STOPPED_MAINTENANCE_REQUIRED | Overall=WARNING
[02 HISTORIAN] Saved row at 2020-06-13 18:01:17 | State=EMERGENCY_STOP_FLAME | Overall=CRITICAL
  
```

Fig. 7. Historian output showing continuous saving of OPC UA snapshots into persistent storage.

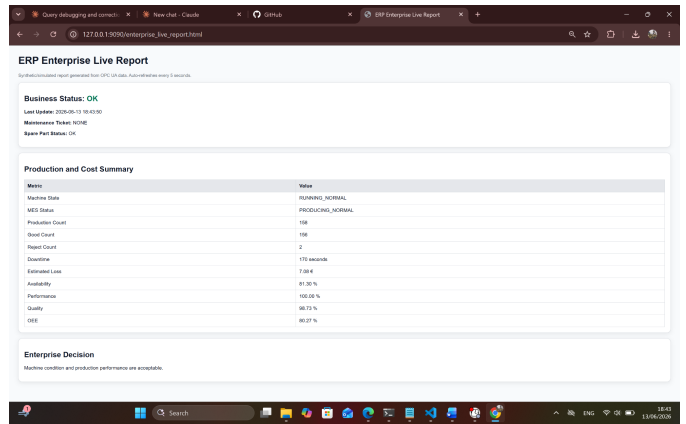


Fig. 8. ERP live report generated from OPC UA values, showing business status, production metrics, loss estimation, and enterprise decision.

XII. CONCLUSION

This project implemented a smart-factory condition monitoring prototype using Raspberry Pi, Arduino sensor nodes, HTTP, MQTT, UDP, and OPC UA. The system successfully receives temperature, flame, and vibration data through

